

Simulatieonderzoek lijnvolgen en obstakeldetectie

Optimale sensorparameters voor de robot-car (eisen F2 en F3.1)

Q-Dat Systems · Project 1 Embedded Systems · 10 september 2026

Opgesteld door **Claude Opus 5** in opdracht van Tygo Poodt. Alle getallen in dit document komen rechtstreeks uit `onderzoek_lijnvolgen.m` en `onderzoek_obstakels.m`.

Onderzoeksvragen

1. *Lijnvolgen (F3.1)*. Hoe ver voor de aandrijfas en met welke onderlinge afstand moeten de vier lijnsensoren zitten, en met welke regelaar volgt de auto de lijn het snelst binnen 5 cm?
2. *Obstakels (F2)*. Welke opstelling van de HC-SR04 en welk gedrag (snelheid, stopafstand, meetfrequentie, filter, remmen, draaien) geven de grootste kans op een botsingsvrije rit van 2 minuten, zo snel mogelijk?

Conclusie. (1) De winst zit in de regelaar, niet in de sensorplaatsing. Met de geoptimaliseerde PD-regelaar ($K_p = 226$ rad/s per m, $v_{\max} = 0,47$ m/s) rijdt de auto het testparcours in 73,0 s in plaats van 124,9 s, met ten hoogste 42 mm afwijking (eis: 50 mm); de huidige regelaar komt tot 62 mm en haalt F3.1 dus niet. De sensorbalk mag overal tussen 45 en 80 mm voor de aandrijfas zitten, met een steek van 8 tot 15 mm; het optimum is $L = 45$ mm en $s = 12,7$ mm ($= w/1,5$: de steek die de lijnpositie in gelijke stappen meet). (2) Een HC-SR04 recht vooruit is voor F2 niet genoeg: hij ziet op de stopafstand maar 58–118 mm van de 143 mm brede neus, en een muur of kist die schuin staat geeft geen echo. Het huidige ontwerp rijdt in 0 % van de gesimuleerde ritten 2 minuten zonder botsing. Beide bestelde HC-SR04's gekruist gebruiken (`kruis2`) werkt duidelijk het best: in een rommelige ruimte 68 % botsingsvrij zonder encoders en 51 % met (95%-interval 41–61 %); dat verschil valt binnen de ruis van de optimalisatie. De kans dat de F2-test (2 van 3 ritten goed) slaagt is daarmee 51 tot 76 %. De grootste onzekerheid is tot welke hoek een vlak nog een echo geeft; die moet eerst aan de echte auto gemeten worden.

1 Opzet: één rekenkern, twee onderzoeken

Beide simulaties gebruiken dezelfde kern (`map model/`), zonder graphics, zodat duizenden ritten parallel kunnen draaien. De 3D-weergave (`demo_*.m`) speelt achteraf een doorgerekende rit af en laadt de carrosserie via `auto_3d_model.m`; een beter 3D-model verandert alleen dat bestand. Alle maten en aannames staan in `qdat_parameters.m`, elk met de bron erbij.

Voertuig. Assenstelsel: oorsprong midden op de aandrijfas, x naar voren, y naar links. Differentiële aandrijving (Siegwart e.a., hoofdstuk 3):

$$v = \frac{1}{2}(v_R + v_L), \quad \omega = \frac{v_R - v_L}{b}, \quad \dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \omega \quad (1)$$

Motor. Per wiel een eerste-ordemodel met dode zone u_{dz} en versterking k_i (accuspanning, verschil links/rechts), begrensd op de grip $a_{\text{slip}} = \mu \sigma g$:

$$\tau_m \frac{dv_i}{dt} = k_i u_{\text{eff},i} v_0 - v_i, \quad u_{\text{eff}} = \text{sgn}(u) \frac{\max(|u| - u_{dz}, 0)}{1 - u_{dz}}, \quad \tau_m = \frac{J_{eq} \omega_0}{\tau_s} = \frac{mr^2 \omega_0}{2\tau_s} \quad (2)$$

Met de motorgegevens uit de frame-analyse ($U_m = 4,25$ V aan de klemmen) is $v_0 = 0,47$ m/s en $\tau_m = 83$ ms. De regelaar stuurt een 8-bits PWM-waarde en compenseert de dode zone met een schatting; de echte dode zone verschilt per run.

Onzekerheid. Een regelaar die alleen werkt in een perfecte simulatie is waardeloos (Jakobi 1995; Tobin 2017). Elke rit krijgt daarom een eigen trekking van alles wat niet met een schuifmaat na te meten is (`variatie_trekken.m`):

grootheid	variatie per rit	waarom
accuspanning (versterking beide motoren)	85–105 %	lege tot volle accu
verschil motor links/rechts	± 5 % (lijnvolgen), ± 1 % (F2, gekalibreerd)	goedkope TT-motoren
dode zone u_{dz}	10–22 % PWM	aanname, meten
motortijdconstante τ_m	80–130 %	wrijving tandwielkast
effectieve spoorbreedte b	± 3 %	contactvlak band
sensorversterking, -offset, -drempel	8 %, 4 %, $\pm 0,08$ per kanaal	potmeters LM393
tapebreedte, breedte lichtvlek	± 1 mm, 80–130 %	
halve openingshoek HC-SR04 β	7,5–15°	datablad noemt 15° en 30°
grootste invalshoek met echo $\gamma_{\max}$	30–50°	aanname, meten
kans op gemiste echo	0–4 %	

Optimaliseren. Er is geen Optimization Toolbox nodig: random search met daarna twee rondes lokale verfijning rond de beste drie kandidaten (**optimalisatie**/). Bij 5 tot 13 parameters werkt dat goed en het laat zich perfect paralleliseren; CMA-ES (Hansen 2016) is overwogen maar niet nodig gebleken. Elke kandidaat rijdt op exact dezelfde banen met dezelfde afwijkingen, zodat verschillen door de instellingen komen en niet door toeval. Het optimum wordt daarna getoetst op banen en ruimtes die de optimizer nooit heeft gezien.

2 Simulatie 1: lijnvolgen (F3.1)

2.1 Sensormodel

Een TCRT5000 kijkt naar een lichtvlek op de vloer. Benader die vlek als een Gaussische vlek met breedte σ ; het deel dat op tape van breedte w valt, als het midden van de vlek op afstand d van de tapehartlijn staat, is

$$c(d) = \frac{1}{2} \left[\operatorname{erf} \left(\frac{w/2 - d}{\sqrt{2}\sigma} \right) + \operatorname{erf} \left(\frac{w/2 + d}{\sqrt{2}\sigma} \right) \right] \quad (3)$$

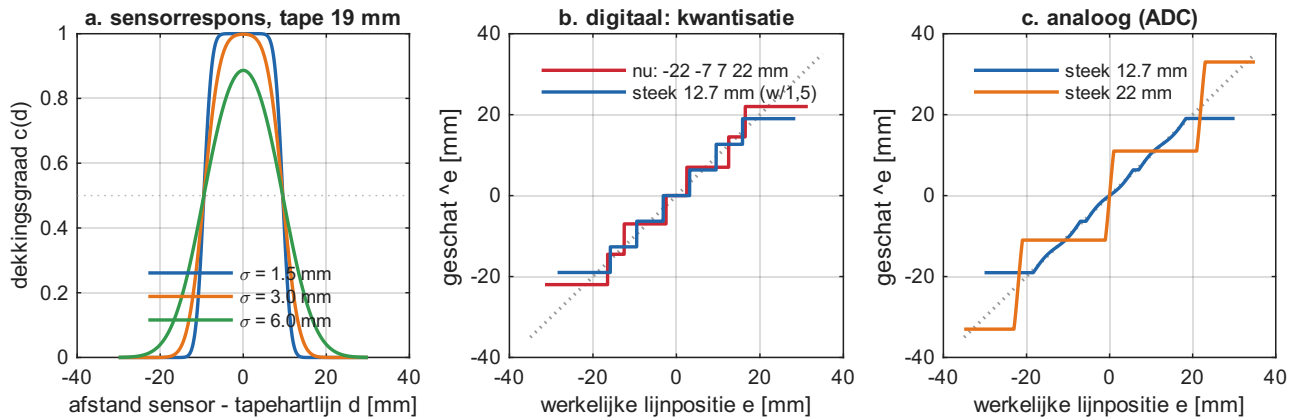
Het signaal is $s_i = g_i c(d_i) + o_i + \text{ruis}$; de comparator van de module maakt daar $b_i = [s_i > \vartheta_i]$ van. De positie van de lijn onder de balk is het zwaartepunt van de actieve sensoren, $\hat{e} = \sum y_i b_i / \sum b_i$. Analooq uitlezen (ADC) gebruikt s_i zelf als gewicht.

2.2 Steek: de lijnpositie in gelijke stappen meten

Bij drempel $\frac{1}{2}$ en $\sigma \ll w$ springt sensor i om bij $e = y_i \pm w/2$. Met $y_i = (i - \frac{N+1}{2})s$ zijn dat twee roosters met steek s , ten opzichte van elkaar verschoven over $w \bmod s$. De stappen in $\hat{e}$ zijn dus afwisselend $w \bmod s$ en $s - (w \bmod s)$, en alleen gelijk ($= s/2$) als

$$\frac{w}{s} = k + \frac{1}{2} \implies s = \frac{w}{1,5} = 12,7 \text{ mm} \quad (k = 1, w = 19 \text{ mm}), \quad e_{\text{bereik}} = \frac{(N-1)s}{2} + \frac{w}{2} \quad (4)$$

Is $s > w$, dan ontstaan er posities waar geen enkele sensor de lijn ziet. De huidige plaatsing ($-22, -7, 7, 22$ mm) geeft stappen van 4 tot 15 mm (figuur 1b).



Figuur 1: (a) Dekkingsgraad (3). (b) Geschatte tegen werkelijke lijnpositie bij digitaal uitlezen: de huidige plaatsing meet in ongelijke stappen, steek $w/1,5$ in gelijke. (c) Analooq uitlezen interpoleert tussen de sensoren.

2.3 Waarom de sensorbalk vóór de as moet zitten: een lineair model

Zij e de zijdelingse afwijking van de aandrijfas ten opzichte van de lijn, ψ de koersfout en κ de kromming van de lijn. Een sensorbalk op afstand L voor de as meet $e_s \approx e + L\psi - \frac{1}{2}L^2\kappa$. Met $\dot{e} = v\psi$, $\dot{\psi} = \omega - v\kappa$ en een P-regelaar $\omega = -Ke_s$ volgt

$$\ddot{e} + KL\dot{e} + Kve = v\kappa \left(\frac{1}{2}KL^2 - v \right) \implies \omega_n = \sqrt{Kv}, \quad \zeta = \frac{L}{2} \sqrt{\frac{K}{v}}, \quad e_{ss} = \kappa \left(\frac{L^2}{2} - \frac{v}{K} \right) \quad (5)$$

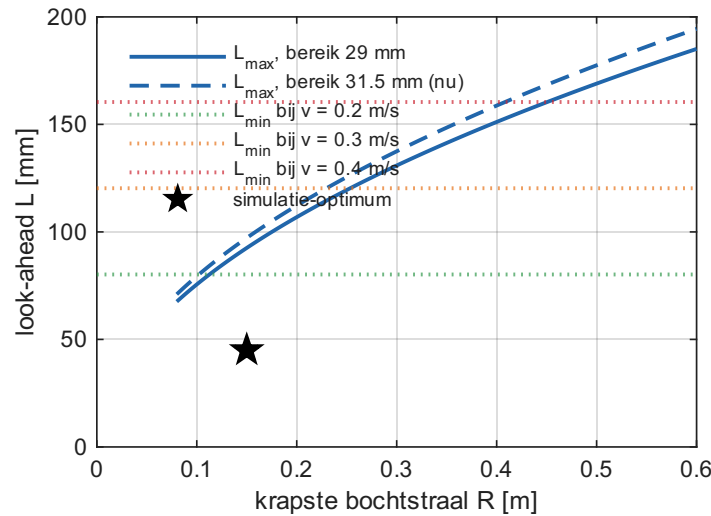
De look-ahead L werkt dus als damping, net als een D-term. De keuze

$$K^* = \frac{2v}{L^2} \implies e_{ss} = 0 \text{ in elke bocht}, \quad \zeta = \frac{1}{\sqrt{2}}, \quad \omega_n = \frac{\sqrt{2}v}{L} \quad (6)$$

geeft tegelijk nul blijvende fout in een bocht én de klassieke optimale damping. Twee grenzen bepalen het venster voor L (figuur 2):

- *bovengrens*: in de krapste bocht (straal R) moet de lijn onder de balk blijven, $L^2/(2R) \leq e_{\text{bereik}}$, dus $L \leq \sqrt{2Re_{\text{bereik}}}$; bij $R = 15$ cm en $e_{\text{bereik}} = 28,5$ mm is dat 92 mm;
- *ondergrens*: de lus mag niet sneller willen reageren dan motor en bemonstering toelaten. Met de totale vertraging $T_d = \tau_m + T_s/2 = 85$ ms en de vuistregel $\omega_n T_d \leq 0,3$ wordt $L \geq \sqrt{2}vT_d/0,3$, bij 0,3 m/s 120 mm.

Bij 0,3 m/s sluiten die grenzen elkaar uit: in krappe bochten moet de auto langzamer. Dat doet de snelheidswet $v = \max(v_{\min}, v_{\max}(1 - k_v|\dot{e}|/e_{\text{bereik}}))$. Daarnaast begrenst de motor zelf de bochtsnelheid: het buitenwiel rijdt $v(1 + b/2R) \leq v_0$, dus $v \leq v_0/(1 + b/2R) = 0,35$ m/s bij $R = 15$ cm. Het simulatie-optimum ($L = 45$ mm) ligt onder de vuistregel-ondergrens: de lus is minder gedempt dan $\zeta = 1/\sqrt{2}$ en slingert licht om de lijn (ongeveer ± 7 mm, figuur 4). De kostfunctie accepteert dat in ruil voor snelheid.



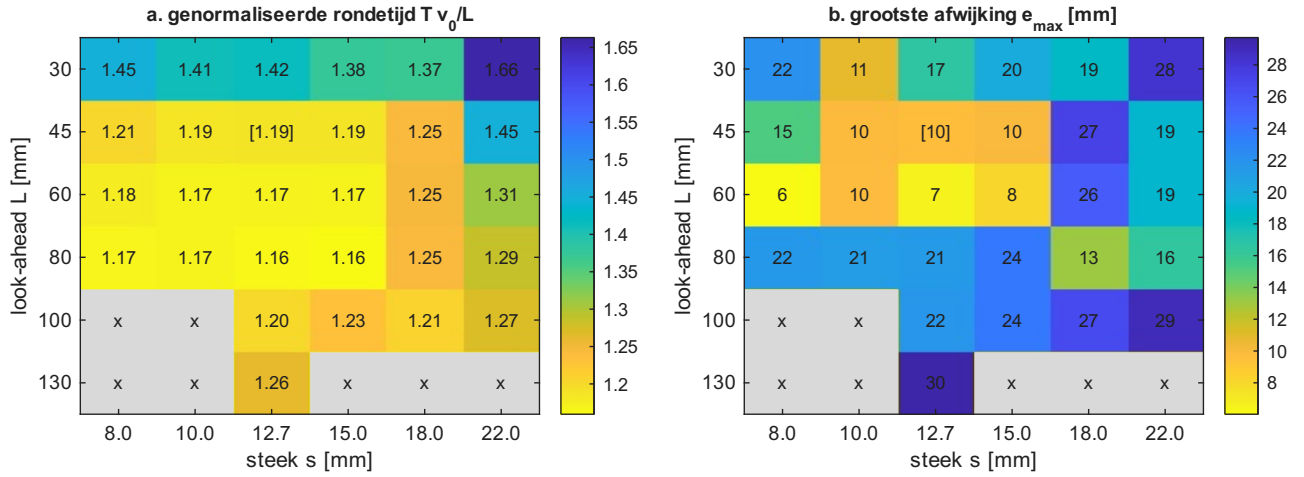
Figuur 2: Venster voor de look-ahead L : boven de blauwe lijn raakt de auto de lijn kwijt in een bocht met straal R , onder een stippellijn reageert de lus te snel voor motor en bemonstering.

2.4 Optimalisatie

Kost per rit (`lijn_kost.m`), lager is beter:

$$J = \frac{Tv_0}{L_{\text{baan}}} + \lambda_e \max\left(0, \frac{e_{\text{max}}}{e_{\text{toel}}} - 1\right) + \lambda_k f_{\text{kwijt}} + \lambda_j j \quad (7)$$

met de genormaliseerde rondetijd ($= 1$: hele ronde op volle motorsnelheid), een straf als de grootste afwijking van de aandrijfas boven $e_{\text{toel}} = 30$ mm komt (F3.1 zegt 50 mm; 20 mm marge voor de verschillen tussen simulatie en werkelijkheid), de tijdfractie dat de lijn kwijt is en het ge jitter van de PWM. Een mislukte rit kost $10 + 10(1 - \text{voortgang})$. Per sensoropstelling ($L \times s$, vier sensoren) worden eerst de vijf regelaarparameters $K_p, K_d, \tau_d, v_{\max}, k_v$ geoptimaliseerd (76 kandidaten, vier trainingsbanen, waaronder een met haakse bochten); pas daarna worden de opstellingen vergeleken. Een opstelling beoordelen met een slecht afgestelde regelaar zegt niets over de opstelling.



Figuur 3: Beste haalbare prestatie per sensoropstelling, elk met een eigen geoptimaliseerde regelaar. Grijs met $\times$: haalt de eis met marge niet. Tussen haken: het optimum.

2.5 Resultaten

Het optimum ligt op $L = 45$ mm en $s = 12,7$ mm, met $K_p = 226$ rad/s per m, $K_d = 0,11$ rad/s per m/s, een D-filter van $\tau_d = 34$ ms, $v_{\max} = 0,47$ m/s en $k_v = 0,68$. Tabel 1 toetst vier instellingen op 10 banen die de optimizer niet kende, elk meerdere keren met andere afwijkingen (30 ritten per instelling).

	voltooid	T testparcours [s]	$\bar{v}$ [m/s]	$e_{\max}$ [mm]	gejitter [PWM/stap]
huidige sim (teken gecorrigeerd)	30/30	124,9	0,25	62	0,459
huidige sensoren, regelaar geoptimaliseerd	30/30	72,8	0,42	43	0,021
optimale sensoren, analytische K^*	30/30	111,6	0,27	12	0,033
optimum (sensoren + regelaar)	30/30	73,0	0,42	42	0,018

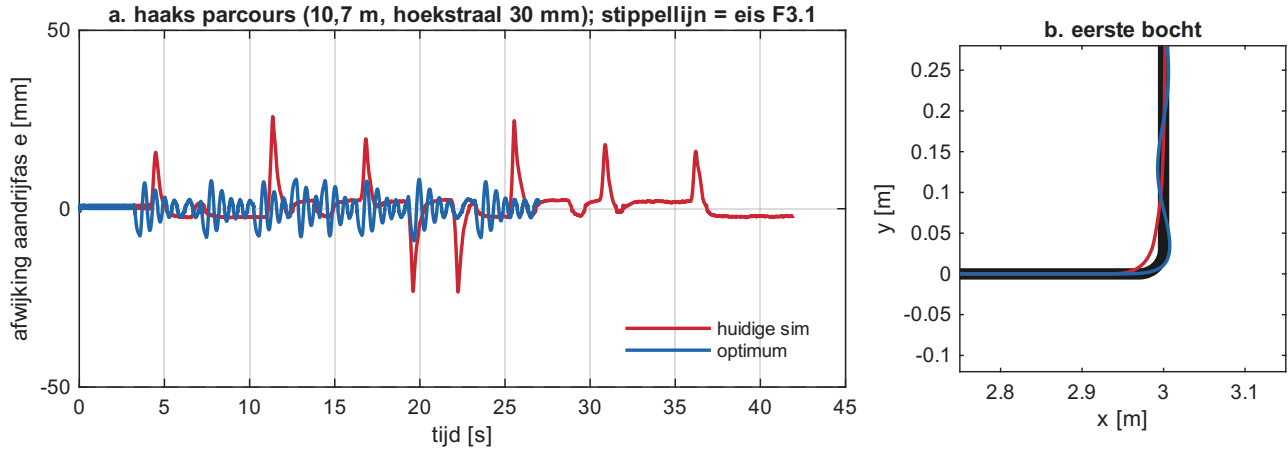
Tabel 1: Validatie. “Huidige sim” is de regelaar uit `RobotCar_LineFollower_Sim.m` ($K_p = 28$ per gewichtseenheid ≈ 2291 rad/s per m, 50 Hz) op het realistische motormodel. Let op: in dat script staat `omega = -(Kp*pos_error + Kd*d_error)` terwijl `pos_error` positief is als de lijn links ligt; zo stuurt de auto van de lijn af (headless nagerekend: na 0,1 s is hij de lijn kwijt en rijdt hij rondjes). Voor deze vergelijking is het teken omgedraaid.

Opvallend: de huidige regelaar komt wel rond, maar zijn K_p is zo groot dat elke sensor die omslaat de wielen van vol links naar vol rechts stuurt (gejitter 0,46 PWM-fractie per stap tegen 0,018 voor het optimum): slecht voor tandwielkast en accu, en traag omdat de auto slingerd. De analytische K^* uit (6) alleen is stabiel maar voorzichtig; de optimizer vindt een hogere snelheid met een snelheidswet die in de bochten terugschakelt.

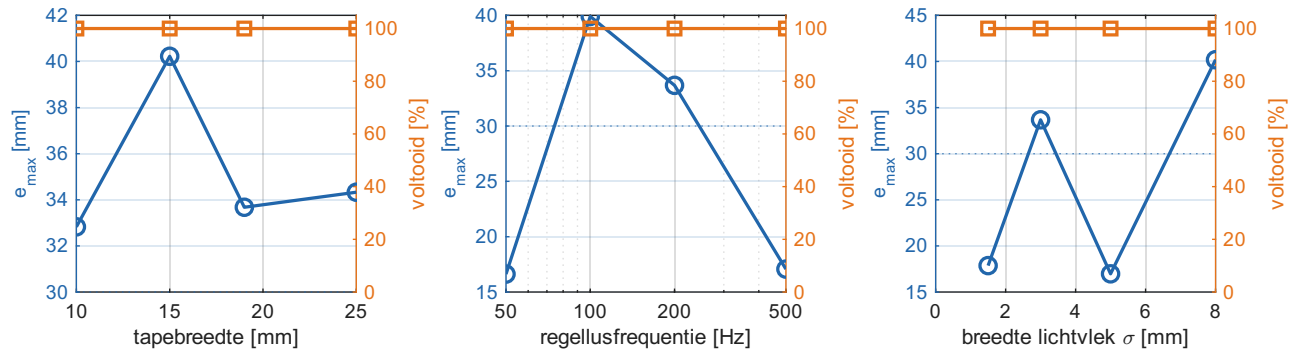
De sensoropstelling zelf maakt binnen een breed gebied weinig uit: in figuur 3 liggen alle opstellingen met L tussen 45 en 80 mm en een steek van 8 tot 15 mm binnen 5 % van elkaar. Pas bij een steek van 18 mm of meer, een balk vlak bij de as (30 mm) of ver ervoor (100 mm en meer) wordt het duidelijk slechter. De winst van 124,9 naar 73,0 s komt dan ook vrijwel helemaal uit de regelaar: met de huidige sensorplaatsing en een geoptimaliseerde regelaar is het resultaat even goed (tabel 1). De huidige regelaar komt wel rond, maar wijkt op de testbanen tot 62 mm af en haalt F3.1 dus niet.

variant (bij $L = 45$ mm)	kost J^*	$T v_0/L$	$e_{\max}$ [mm]
2 sensoren	1,44	1,25	10
3 sensoren	1,36	1,21	12
4 sensoren (optimum)	1,31	1,19	10
5 sensoren	1,29	1,19	11
4 sensoren, analoog	1,24	1,18	11
4, analoog, steek 22 mm	1,40	1,25	17

Tabel 2: Aantal sensoren en analoog uitlezen, elk met een eigen geoptimaliseerde regelaar.



Figuur 4: Huidige sim tegen optimum op een parcours met haakse bochten.



Figuur 5: Robuustheid van het optimum op de testbanen. Links: tapebreedte (T9.1 staat 1 tot 2 cm toe). Bij 10 mm is de steek groter dan de tape, zodat volgens (4) gaten ontstaan; de auto komt toch rond ($e_{\max} = 33$ mm, 100 % voltooid), omdat de lijn door de look-ahead en de lichtvlek zelden lang in een gat blijft. Midden: regellusfrequentie. Rechts: breedte van de lichtvlek (hoogte van de sensor). $e_{\max}$ is een slechtste-gevalwaarde over een beperkt aantal ritten en springt daardoor; geen van de drie laat een duidelijke trend zien.

2.6 Vertaling naar de Arduino-code

De regelaar werkt in SI-eenheden; op de ATmega328P (geen FPU) wordt het integerwerk (Wescott 2000). Per millimeter lijnfout verschuift elk wiel $\Delta \text{PWM} = K_p \cdot 10^{-3} \cdot \frac{b}{2} / v_0 \cdot 255 \approx 6,6$ PWM-stappen (links erbij, rechts eraf). De D-term op de gemeten positie door een eerste-ordefilter: $D_k = D_{k-1} + \frac{T_s}{\tau_d + T_s} ((\hat{e}_k - \hat{e}_{k-1}) / T_s - D_{k-1})$; reken $\hat{e}$ in hele millimeters en schaal de versterkingen met een macht van 2, dan blijft alles in `int16_t`.

3 Simulatie 2: obstakels herkennen en ontwijken (F2)

3.1 Model van de HC-SR04

De module meet de looptijd van de eerste echo die terugkomt:

$$d = \frac{c t_{\text{echo}}}{2}, \quad c = 331,3 + 0,606 T \text{ m/s} \quad (T \text{ in } ^\circ\text{C}) \quad (8)$$

In de simulatie is de bundel een waaier van stralen over $[-\beta, \beta]$. Een straal telt alleen als het vlak dat hij raakt hem terugkaatst: de invalshoek (tussen straal en normaal) moet kleiner zijn dan $\gamma_{\max}$. Een glad vlak dat te schuin staat kaatst de puls weg, als een spiegel (Leonard & Durrant-Whyte 1992); kistranden en binnenhoeken geven wel altijd een zwakke echo. Daarna komen ruis ($\sigma = 3$ mm), afronding op hele centimeters zoals de Arduino-code doet, 0–4 % gemiste echo's, en de timing: een puls om de $T_{\text{meas}} \geq 60$ ms (datablad), het resultaat $2d/c$ later, bij geen echo pas na de time-out van 38 ms. Twee of drie sensoren vuren om de beurt, anders horen ze elkaars echo.

3.2 Remweg en de hoogste snelheid die F2.3 haalt

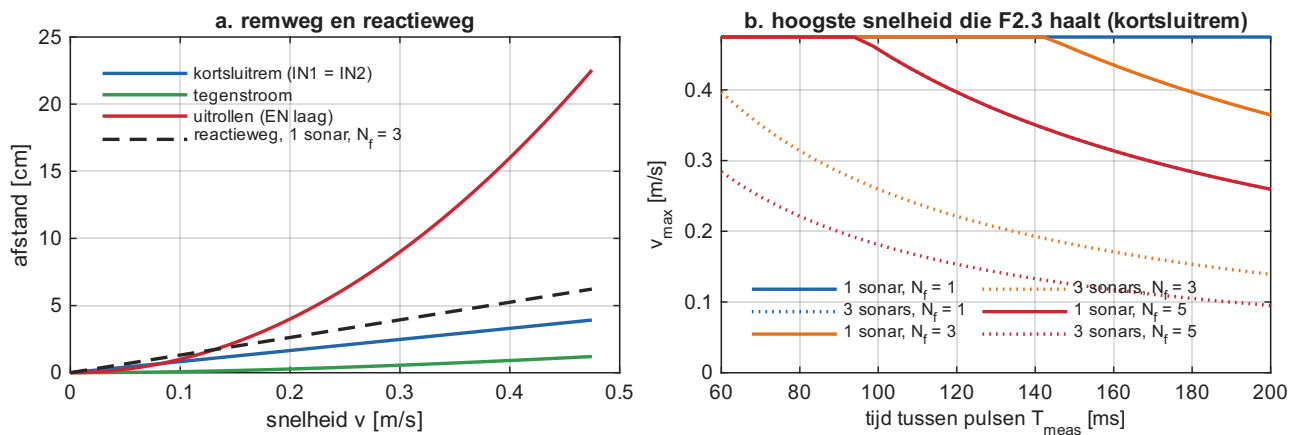
Van “obstakel dichters dan d_{stop} ” tot “remmen” zit in het slechtste geval

$$t_{\text{lat}} = n T_{\text{meas}} \left(1 + \frac{N_f - 1}{2} \right) + \frac{2d}{c} + \frac{1}{f_s} \quad (9)$$

(n sensoren om de beurt; een mediaanfilter over N_f metingen heeft $(N_f + 1)/2$ lage metingen nodig). Voor één sensor, 60 ms en $N_f = 3$ is dat 131 ms, voor drie sensoren 411 ms. De remweg hangt af van hoe de L298N remt:

$$s_{\text{rem}} = \begin{cases} v \tau_m & \text{kortsluitrem (IN1 = IN2, EN hoog)} \\ v \tau_m - v_0 \tau_m \ln(1 + v/v_0) & \text{tegenstroom, gedurende } t^* = \tau_m \ln(1 + v/v_0) \\ v^2 / (2a_{\text{rol}}) & \text{uitrollen (EN laag)} \end{cases} \quad (10)$$

Bij 0,3 m/s is dat 2,5, 0,6 en 9,0 cm. F2.3 eist remmen vanaf 20 cm met 2 cm speling, dus $v t_{\text{lat}} + s_{\text{rem}}(v) \leq 18$ cm. Met de kortsluitrem geeft dat $v_{\text{max}} = 0,47$ m/s voor één sensor en 0,36 m/s voor drie; laat de code de motoren uitrollen (`analogWrite(EN, 0)`), dan nog maar 0,36 m/s (figuur 6).



Figuur 6: (a) Remweg per remmethode en reactieweg. (b) Hoogste snelheid waarbij de auto volgens F2.3 op tijd stilstaat, als functie van de meetperiode en de filterlengte.

3.3 Welk deel van de auto ziet de sonar?

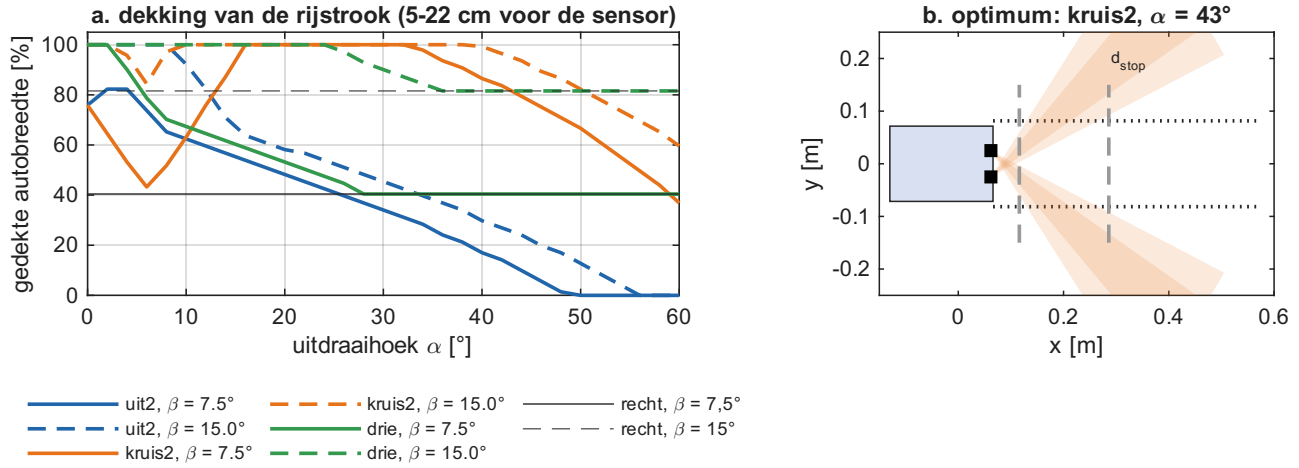
Een obstakel op zijafstand y ligt in de bundel van één sensor recht vooruit zolang $x \geq |y|/\tan\beta$. Nadert de auto, dan valt het er juist *uit*. Op de stopafstand is de beschermde breedte

$$B_{\text{besch}} = 2 d_{\text{stop}} \tan\beta \quad \text{en de hele breedte } B \text{ vraagt} \quad d_{\text{stop}} \geq \frac{B/2}{\tan\beta} \quad (11)$$

Bij $d_{\text{stop}} = 22$ cm is $B_{\text{besch}} = 58$ mm ($\beta = 7,5^\circ$) tot 118 mm ($\beta = 15^\circ$) van de 143 mm; de hele breedte zien vraagt stoppen op 54 resp. 27 cm, en dat botst met F2.1 (doorrijden zolang 30 cm vrij is). Een poot of de hoek van een kist die 4–7 cm naast het midden staat, raakt dus de neus zonder ooit gezien te zijn. Voor meer sensoren geldt: een punt (x, y) wordt door sensor i op (x_i, y_i) met kijkrichting α_i gezien als

$$|\text{atan2}(y - y_i, x - x_i) - \alpha_i| \leq \beta \quad (12)$$

Figuur 7a telt voor elke opstelling welk deel van de breedte ergens tussen 5 en 22 cm voor de sensor in een bundel valt. Twee sensoren “scheel” (de linker kijkt naar rechts, `kruis2`) dekken het meest: bij elke α tussen 16° en 32° valt 100 % van de breedte in een bundel, tegen 40–82 % voor één sensor. Twee naar buiten gedraaide sensoren (`uit2`) laten het midden open.



Figuur 7: (a) Gedekte autobreedte als functie van de uitdraaihoek, uit (12). (b) Bundels van het simulatie-optimum ($\beta = 7,5^\circ$ en 15°); stippellijnen: rijstrook.

3.4 Rechtdoor rijden, draaien en de achterkant

Drift (F2.1). Draait het rechterwiel een fractie δ sneller dan gevraagd en het linker δ langzamer, dan rijdt de auto een cirkel met straal $b/(2\delta)$ en kijkt hij over een afstand s af met

$$y(s) = \frac{\delta s^2}{b} \quad \Rightarrow \quad \delta \leq \frac{y_{\max} b}{s^2} = \frac{0,40 \cdot 0,108}{2^2} = 1,1 \% \quad (13)$$

Zonder terugkoppeling moeten de motoren dus tot op ongeveer 1 % gelijk lopen; ongekalibreerde TT-motoren (± 5 %) rijden een cirkel van 1,1 m. Een trimfactor per motor (afregelen op een rechte lijn) of encoders zijn nodig. Vanaf hier is aangenomen dat de motoren op 1 % na zijn gekalibreerd.

Draaien zonder encoders. Zonder encoders weet de regelaar zijn draaihoek alleen uit tijd maal gevraagde draaisnelheid. Bij 2 rad/s is de PWM-fractie per wiel $u = 23$ %; na dode-zonecompensatie met een geschatte $\hat{u}_{dz}$ is de echte draaisnelheid

$$\frac{\omega_{\text{echt}}}{\omega_{\text{gevraagd}}} = \frac{u' - u_{dz}}{(1 - u_{dz})u}, \quad u' = \hat{u}_{dz} + (1 - \hat{u}_{dz})u \quad (14)$$

wat voor u_{dz} tussen 22 en 10 % een fout van -30 tot +19 % geeft. “Kijk links en rechts en draai terug naar het beste gat” mist daardoor tientallen graden; “draai tot de sonar vrij meldt” niet, want die sluit de lus via de sensor. Met encoders van 20 pulsen per omwenteling is de hoekstap $2\pi r/(Nb) = 5,3^\circ$.

Draaisnelheid. Om geen gat te missen mag de auto tussen twee metingen van dezelfde sensor niet verder draaien dan de bundelbreedte: $\omega n T_{\text{meas}} \leq 2\beta$, dus $\omega \leq 4,4$ rad/s voor één sensor en 1,3 rad/s voor drie.

Achterkant. Op de plaats draaien zwiept de achterkant rond met straal $R = \sqrt{a^2 + (B/2)^2} = 148$ mm (a = overhang achter de as), veel meer dan de neus (97 mm). De sonar ziet dat niet. Met een overhang van 90 mm (de frame-analyse) wordt dat 115 mm; in de simulatie gaat het botsingsvrije percentage daarmee van 51 naar 55 %. Een input voor het verbeterde 3D-model.

3.5 Gedrag en optimalisatie

De toestandsmachine volgt F2.1–F2.4: RIJDEN (vol door zolang de rijstrook $\geq d_{\text{slow}}$ vrij is, daarna afbouwen tot v_{kruip}), REMMEN onder d_{stop} , KIEZEN (op de plaats draaien tot er 60 cm vrij is, of eerst naar beide kanten kijken) en zo nodig ACHTERUIT (max. 30 cm). Drie “slimme” regels die de simulatie nodig bleek te hebben:

- *rijstrook-logica*: een echo telt alleen als hij binnen de breedte van de auto kan liggen, zodat een schuine sensor niet stopt voor een muur waar de auto langs rijdt;
- *geen echo is niet vrij*: tijdens het kiezen telt een time-out niet als vrije ruimte, want dat is precies wat een schuine muur geeft;
- *doordraaien*: na de eerste vrije meting nog φ_{extra} verder draaien, zodat de auto niet vertrekt langs de rand van het gat.

Per opstelling (recht, kruis2, uit2, drie), met en zonder encoders, zoekt de optimizer 13 parameters (obst_zet.m) af in 16 arena’s, met kost

$$J = f_{\text{botsing}} + 0,25 \left(1 - \frac{\bar{s}}{v_0 T_{\text{rit}}} \right) + 0,1 f_{\text{eis}} \quad (15)$$

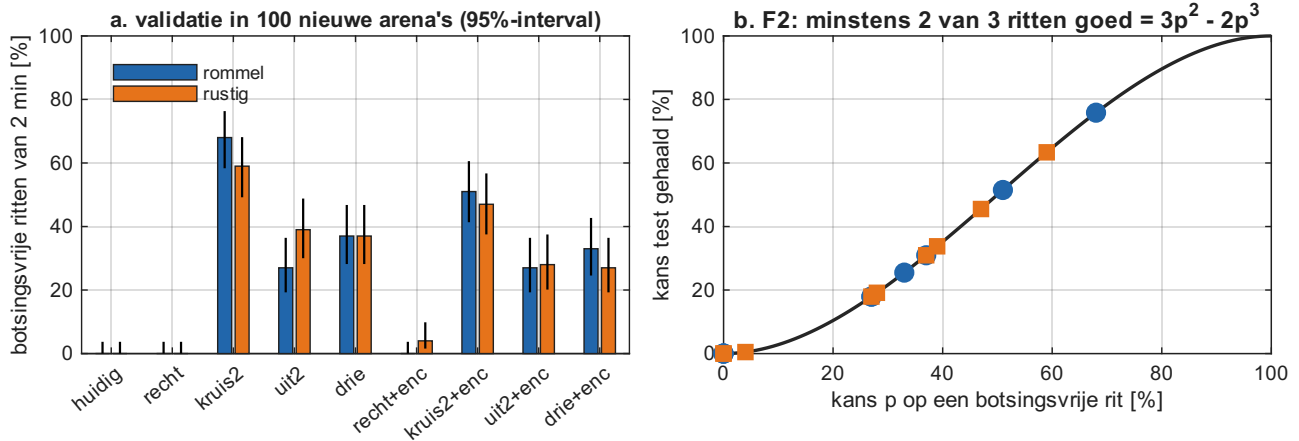
(fractie ritten met een botsing, gemiddeld afgelegde afstand, fractie stops die F2.3 of F2.4 schendt). Daarna rijdt elk optimum in 100 nieuwe arena's van twee soorten: *rommel* (kisten onder willekeurige hoek en dunne poten) en *rustig* (alleen kisten van minstens 15 cm).

3.6 Resultaten

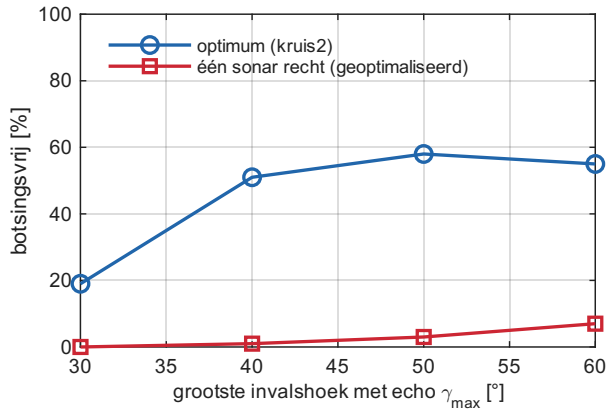
opstelling	koers	botsingsvrij rommel	botsingsvrij rustig	test gehaald	afstand [m/2 min]	fout F2.3/2.4 per stop
huidig (1 recht, ongekal.)	tijd	0 %	0 %	0 %	1,5	3 %
recht	tijd	0 %	0 %	0 %	3,3	3 %
kruis2	tijd	68 %	59 %	76 %	9,0	6 %
uit2	tijd	27 %	39 %	18 %	4,4	23 %
drie	tijd	37 %	37 %	31 %	9,2	3 %
recht	encoders	0 %	4 %	0 %	5,2	2 %
kruis2	encoders	51 %	47 %	51 %	8,7	9 %
uit2	encoders	27 %	28 %	18 %	4,1	45 %
drie	encoders	33 %	27 %	25 %	4,7	25 %

Tabel 3: Validatie per opstelling, elk met eigen geoptimaliseerd gedrag. “Test gehaald” is de kans dat minstens 2 van 3 ritten botsingsvrij zijn: $3p^2 - 2p^3$.

Op de trainingsarena's scoorde **kruis2** met encoders het best, met uitdraaihoek 43° , kruissnelheid 0,14 m/s, stoppen op 28 cm en afbouwen vanaf 29 cm, een puls om de 69 ms, mediaanfilter $N_f = 1$, draaien met 1,2 rad/s (strategie *draai*, doordraaien 46°), remmen met *tegenstroom* en “geen echo is niet vrij”: nee. In de validatie deed dezelfde opstelling *zonder* encoders het beter (68 tegen 51 %). Met 16 trainingsarena's is de keuze van de optimizer tussen twee zo dicht bij elkaar liggende varianten dus niet betrouwbaar. Wat wel vaststaat: **kruis2** is de enige opstelling die ruim boven de andere uitkomt, en één sonar recht vooruit haalt vrijwel nooit een rit van 2 minuten. De optimizer koos een lage kruissnelheid en een grote uitdraaihoek (43° , buiten het plateau van figuur 7a): hij ruilt dekking van de hoeken van de auto in voor het zien van muren onder een scherende hoek. Alle instellingen per opstelling staan in **resultaten/obst_optimum.mat** (variabele RES).

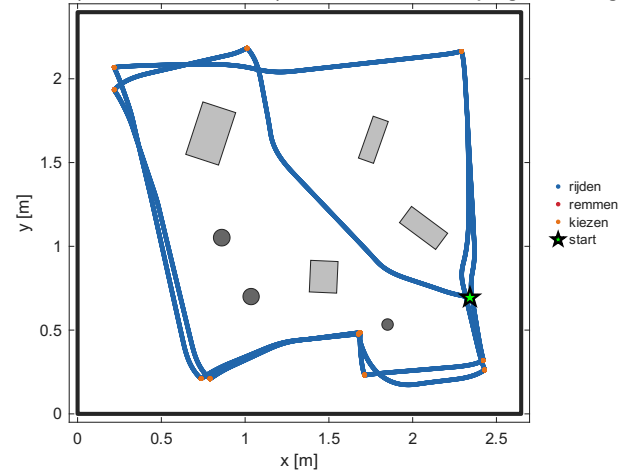


Figuur 8: (a) Botsingsvrije ritten per opstelling in nieuwe arena's. (b) Van kans per rit naar kans dat de F2-test slaagt.



Figuur 9: De uitkomst hangt sterk af van $\gamma_{\max}$: van 19 % bij 30° tot 55 % bij 60° voor het optimum.

arena 1024 (2.6 x 2.4 m, 7 obstakels): 16.0 m in 120 s, 11 stops, geen botsing



Figuur 10: Een rit van het optimum; kleur = toestand.

4 Wat dit betekent voor het ontwerp

1. **Lijnsensor:** vier kopjes op een steek van 12,7 mm, 45 mm voor de aandrijfas; alles tussen 45 en 80 mm en een steek van 8 tot 15 mm is vrijwel even goed. De losse kopjes van een 4-kanaals tracker kun je vrij plaatsen. Wordt de tape 10 mm, zet de steek dan op $w/1,5 \approx 7$ mm.
2. **Lijnregelaar:** dit is waar de winst zit. PD met gefilterde D-term op 200 Hz, instellingen hierboven; begin op de echte auto met deze waarden en stel alleen K_p en $v_{\max}$ bij. Wie voorzichtig wil beginnen: de analytische $K^* = 2v/L^2$ geeft de kleinste afwijking (12 mm), maar is anderhalf keer zo traag.
3. **Sonar:** gebruik beide bestelde HC-SR04's voor F2, gekruist: de linker kijkt 16–32° naar rechts en omgekeerd (de simulatie koos 43°). Laat ze om de beurt vuren.
4. **Motoren:** kalibreer een trimfactor per motor (F2.1 vraagt $\leq 1,1$ % verschil). Encoders gaven in deze simulatie geen aantoonbare winst voor F2 (tabel 3), maar maken draaien over een vaste hoek wel nauwkeurig.
5. **Remmen:** kortsluitrem (IN1 = IN2 met EN hoog), nooit uitrollen.
6. **3D-model:** maak de overhang achter de as korter of de achterkant rond (zwaaistraal).

5 Wat eerst gemeten moet worden

Een simulatie is zo goed als de getallen die erin gaan (Mouret & Chatzilygeroudis 2017). De getallen met [AANN] in `qdat_parameters.m` zijn schattingen; de eerste vier zijn het belangrijkste:

1. $\gamma_{\max}$: zet de auto op 30 cm voor de muur en een kist, draai hem in stappen van 5° en noteer tot welke hoek de HC-SR04 nog een echo geeft. Dit getal bepaalt de uitkomst van F2 het meest (figuur 9).
2. Dode zone en v_0 : laat de PWM langzaam oplopen tot de wielen draaien; meet de snelheid bij PWM 255 over 2 m.
3. τ_m en remweg: film een noodstop vanaf 0,3 m/s naast een meetlat; vergelijk met (10).
4. Lichtvlek σ : schuif een sensorkopje dwars over de tape en log de analoge waarde; pas (3) erop.

Het eigenlijke resultaat van dit onderzoek is dan het verschil tussen simulatie en werkelijkheid in meetbare eenheden: rijd hetzelfde parcours met dezelfde instellingen en vergelijk rondetijd en $e_{\max}$ met tabel 1.

6 Bestanden

bestand	functie
<code>qdat_parameters.m</code>	alle maten en aannames, met bron
<code>onderzoek_lijnvolgen.m</code>	simulatie 1: kaart, varianten, validatie, figuren (58 min)
<code>onderzoek_obstakels.m</code>	simulatie 2: analyse, optimalisatie, validatie, figuren (32 min)
<code>demo_lijnvolgen.m, demo_obstakels.m</code>	3D-weergave van een rit, optioneel als MP4
<code>model/</code>	rekenkern: voertuig, sensoren, banen, arena's, beide simulaties, 3D-model
<code>optimalisatie/</code>	kostfuncties, random search, validatie
<code>resultaten/</code>	figuren en .tex-bestanden met de getallen voor dit document

Referenties

- [1] V. Braitenberg, *Vehicles: Experiments in Synthetic Psychology*. MIT Press, 1984.
- [2] N. Hansen, “The CMA Evolution Strategy: A Tutorial”, arXiv:1604.00772, 2016.
- [3] N. Jakobi, P. Husbands, I. Harvey, “Noise and the Reality Gap”, ECAL 1995, LNCS 929, pp. 704–720.
- [4] J. J. Leonard, H. F. Durrant-Whyte, *Directed Sonar Sensing for Mobile Robot Navigation*. Kluwer, 1992.
- [5] J.-B. Mouret, K. Chatzilygeroudis, “20 Years of Reality Gap”, GECCO '17 Companion, pp. 1121–1124.
- [6] R. Siegwart, I. R. Nourbakhsh, D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2e dr. MIT Press, 2011.
- [7] J. Tobin e.a., “Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World”, IROS 2017, pp. 23–30.
- [8] T. Wescott, “PID Without a PhD”, *Embedded Systems Programming*, okt. 2000.